

UniVia

Plan de Trabajo — Culminación de Backend

Cierre de los requerimientos funcionales pendientes en la API de FastAPI, en espejo con el plan de rebrand de frontend, distribuido en 4 fases asignadas a Luis, Renato, Sebastián y Omar.

Proyecto: leaduni/univia-project

Alcance: Backend (backend/app — FastAPI, Supabase/Postgres, módulo RAG)

Fuente de requerimientos: UNIVIA - REQUERIMIENTOS.pdf (Doc A) y

Requerimientos_Plataforma_Estudiantes_UNI.pdf (Doc B)

Documento espejo de: UniVia_Plan_Rebrand_Frontend.pdf

Equipo: Luis · Renato · Sebastián · Omar

Fecha de elaboración: Julio 2026

1. Contexto y objetivo

El plan de rebrand de frontend divide el trabajo en 4 módulos: **(1) Fundación visual**, **(2) Auth/Onboarding**, **(3) Dashboard/Navegación/Perfil** y **(4) Cursos/Evaluaciones con IA/Recursos**. El backend ya tiene un esqueleto de routers que corresponde casi 1 a 1 con esos mismos módulos (`backend/app/routers/onboarding.py`, `usuarios.py`, `dashboard.py`, `mall.py`, `courses.py`, `evaluaciones.py`, más el módulo `backend/app/rag/`), pero varios requerimientos funcionales (RF) descritos en los documentos de requerimientos aún no están completamente implementados o verificados.

Este plan asigna la **culminación del backend** a las mismas 4 personas y en el mismo orden de dependencias que el rebrand de frontend, de modo que cada integrante cierre el backend del mismo módulo cuyo frontend está rediseñando. Esto evita coordinación cruzada innecesaria: quien redefine la pantalla también termina el contrato de datos que la alimenta.

Fuentes de requerimientos

Documento A — *UNIVIA - REQUERIMIENTOS.pdf*: requerimientos funcionales con prefijos RF-ALC (alcance), RF-EST (estudiantes/registro), RF-APR (rutas de aprendizaje y evaluaciones) y RF-PRF (perfil).

Documento B — *Requerimientos_Plataforma_Estudiantes_UNI.pdf*: requerimientos funcionales numerados RF-01 a RF-22 y no funcionales RNF-01 a RNF-10.

Principio de trabajo en ramas

Cada fase se desarrolla en su propia rama (`feature/backend-<fase>`), partiendo siempre de `main` actualizado. Ninguna fase se mergea a `main` sin antes sincronizar con las fases de las que depende. El orden de merge sigue el orden de dependencias de la sección 3.

2. Resumen de asignaciones

Fase	Responsable	Rama	Enfoque backend	Módulo frontend equivalente	Dependencia
Fase 1	Luis	<i>feature/backend-foundation</i>	Núcleo de datos y seguridad: esquema de BD, autenticación base, validaciones transversales	Fase 1 rebrand — Fundación visual	Ninguna (arranca primero)
Fase 2	Renato	<i>feature/backend-auth-onboarding</i>	Registro, login, recuperación de contraseña, onboarding (carrera/ciclo/cursos/prerrequisitos)	Fase 2 rebrand — Auth/Onboarding	Requiere Fase 1. Paralelo con Fase 3
Fase 3	Sebastián	<i>feature/backend-core-app</i>	Malla curricular, avance académico, dashboard con estadísticas, gestión de perfil	Fase 3 rebrand — Dashboard/Navegación/Perfil	Requiere Fase 1. Paralelo con Fase 2
Fase 4	Omar	<i>feature/backend-courses-evaluations</i>	Cursos y rutas de aprendizaje, evaluaciones con IA, reforzamiento con IA, banco de recursos	Fase 4 rebrand — Cursos/Evaluaciones IA/Recursos	Requiere Fase 1. Consume contrato de Fase 3

3. Mapa de dependencias entre fases

El siguiente esquema resume qué fases pueden avanzar de forma independiente y cuáles requieren que otra fase termine antes de integrarse a main:

FASE 1 (Luis) — BLOQUEANTE PARA TODAS

Debe completarse y mergearse a main primero. Define el esquema de base de datos, las utilidades de autenticación (core/auth_utils.py, core/database.py) y las validaciones transversales (formato de correo institucional, unicidad de código de alumno, cifrado de credenciales) que las demás fases van a consumir. Ninguna otra fase puede darse por terminada sin este trabajo integrado.

FASE 2 (Renato) y FASE 3 (Sebastián) — EN PARALELO ENTRE SÍ

Una vez mergeada la Fase 1, Renato y Sebastián pueden trabajar simultáneamente: Renato en autenticación/registro/onboarding (routers/onboarding.py, routers/usuarios.py — login y recuperación), Sebastián en malla curricular, dashboard y perfil (routers/malla.py, routers/dashboard.py, routers/usuarios.py — actualización de perfil). No comparten endpoints exclusivos entre sí, solo el núcleo de Fase 1.

FASE 4 (Omar) — PARCIALMENTE DEPENDIENTE DE FASE 3

Omar puede iniciar el diseño de rutas de aprendizaje y el banco de recursos en paralelo con las Fases 2 y 3, pero la generación de evaluaciones con IA y el reforzamiento (RF-APR-04, RF-APR-05, RF-15 a RF-18) necesitan el modelo de avance/perfil del estudiante que Sebastián expone en la Fase 3 para poder personalizar el contenido generado. Antes del cierre final de la Fase 4, Omar debe sincronizar su rama con la Fase 3 ya mergeada.

Orden de merge recomendado a main: Fase 1 → (Fase 2 y Fase 3 en cualquier orden, una vez listas) → Fase 4 (al final, tras sincronizar con Fase 3).

Detalle de fases

FASE 1

Núcleo de datos, seguridad y validaciones transversales

Responsable: Luis | Rama: feature/backend-foundation

SIN DEPENDENCIAS — Esta fase se inicia de inmediato. Es **BLOQUEANTE**: ninguna otra fase se considera lista para producción hasta que esta esté mergeada a main.

Luis establece la base de datos y seguridad sobre la que trabajarán los demás: esquema de tablas (estudiantes, carreras, cursos, malla, evaluaciones), utilidades de autenticación y las reglas de validación que se repiten en varios módulos.

RF cubiertos: RF-ALC-02 (criterio de plataforma lista), RF-EST-01, RF-EST-02 (formato de correo institucional y unicidad de código), RNF-04 (cifrado de contraseñas/datos personales), RNF-05 (escalabilidad de esquema), RNF-07 (mantenibilidad), RNF-09 (privacidad académica).

Subtareas (en orden)

- 1** Auditoría del esquema actual en `core/database.py`: mapear tablas existentes (estudiantes, cursos, malla, evaluaciones) contra los datos mínimos de perfil de RF-EST-01 (código universitario, correo institucional, nombres y apellidos, carrera, ciclo relativo, lista de cursos) y completar columnas/tablas faltantes.
Archivos/carpetas: `backend/app/core/database.py`
- 2** Implementar las validaciones de formato de RF-EST-02: verificación de dominio `@uni.pe` en el correo y restricción de unicidad del código universitario a nivel de base de datos y de servicio, con mensajes de error reutilizables por los demás módulos.
Archivos/carpetas: `backend/app/core/auth_utils.py`
- 3** Definir el modelo de prerequisites de cursos (tabla o relación `curso→prerequisites`) que sustente RF-EST-03 (deshabilitar selección de cursos sin prerequisites cumplidos), para que Renato lo consuma en el onboarding sin tener que diseñarlo desde cero.
Archivos/carpetas: `backend/app/core/database.py`
- 4** Reforzar `auth_utils.py` con hashing/cifrado de contraseñas y sesiones acorde a RNF-04, y documentar las condiciones de seguridad de contraseña (cantidad de caracteres y reglas) que luego usará RF-PRF-03.
Archivos/carpetas: `backend/app/core/auth_utils.py`
- 5** Configurar el punto de entrada de la API (`main.py`): CORS, manejo de errores centralizado, estructura de respuesta estándar y variables de entorno (`.env.example`) para que Renato, Sebastián y Omar no dupliquen configuración en sus routers.
Archivos/carpetas: `backend/app/main.py`, `backend/.env.example`
- 6** Publicar una guía corta de esquema y convenciones (qué tabla/columna usar para qué caso, cómo extender el modelo de curso/malla) para que el resto del equipo no improvise estructuras de datos paralelas.
Archivos/carpetas: `documentacion/backend/esquema-datos.md` (nuevo)

Criterio de fase terminada (Definition of Done)

- Esquema de base de datos cubre los datos mínimos de perfil (RF-EST-01) y el modelo de prerequisites (RF-EST-03).
- Validaciones de correo institucional y unicidad de código (RF-EST-02) están implementadas y son reutilizables desde cualquier router.

- Contraseñas y datos personales se almacenan cifrados (RNF-04).
- Existe documentación de esquema que las Fases 2, 3 y 4 pueden consultar.
- Rama mergeada a main y comunicada al equipo antes de que las demás fases hagan merge.

FASE 2

Registro, autenticación y onboarding

Responsable: Renato | Rama: feature/backend-auth-onboarding

DEPENDENCIA DE FASE 1 (mergeada). Puede avanzar en PARALELO con Fase 3. No depende de Fase 4.

Renato completa el flujo de entrada a la plataforma en el backend: login, recuperación de contraseña, registro de perfil y el onboarding de selección de carrera/ciclo/cursos, en correspondencia con el flujo de login/signup/wizard que rediseña en el frontend.

RF cubiertos: RF-01, RF-02, RF-03 (login, asociación automática de carrera/plan, recuperación de contraseña), RF-EST-01, RF-EST-02, RF-EST-03 (registro y validaciones), RF-PRF-01 (actualización de cursos por ciclo).

Subtareas (en orden)

- 1 Sincronizar la rama con main (post Fase 1) y completar en routers/usuarios.py el endpoint de login con código de alumno/correo institucional y contraseña (RF-01), devolviendo la carrera y plan de estudios asociados automáticamente (RF-02).
Archivos/carpetas: backend/app/routers/usuarios.py
- 2 Implementar el flujo de recuperación/restablecimiento autónomo de contraseña (RF-03): solicitud de reseteo, token temporal y endpoint de actualización, reutilizando las reglas de seguridad de contraseña definidas en la Fase 1.
Archivos/carpetas: backend/app/routers/usuarios.py
- 3 Completar en routers/onboarding.py el endpoint de registro de perfil con los datos mínimos de RF-EST-01, aplicando las validaciones de formato/unicidad de la Fase 1 (RF-EST-02).
Archivos/carpetas: backend/app/routers/onboarding.py
- 4 Implementar la lógica de prerequisites de RF-EST-03 sobre el modelo definido por Luis: deshabilitar/filtrar en la respuesta del API los cursos cuyo prerequisite no esté aprobado por el estudiante, para que el wizard de onboarding (paso 'Academic Status Step') reciba la lista ya filtrada.
Archivos/carpetas: backend/app/routers/onboarding.py
- 5 Implementar el endpoint de selección de carrera y ciclo relativo que alimenta los pasos 'Career Step' y 'Semester Step' del wizard, devolviendo catálogo de carreras/facultades y rango de ciclos disponible.
Archivos/carpetas: backend/app/routers/onboarding.py
- 6 Implementar RF-PRF-01 (actualización de lista de cursos por ciclo): endpoint que permite al estudiante reemplazar su lista de cursos activos cada inicio de ciclo, sin perder el historial de cursos aprobados usado por la malla (Fase 3).
Archivos/carpetas: backend/app/routers/onboarding.py, backend/app/routers/usuarios.py
- 7 Implementar el endpoint de resumen de finalización de onboarding ('Completion Step'): cursos aprobados, cursos disponibles y porcentaje de avance de carrera, como precálculo que después reutiliza el dashboard de la Fase 3.
Archivos/carpetas: backend/app/routers/onboarding.py
- 8 Pruebas manuales de extremo a extremo del flujo login → registro → onboarding con casos de error (correo fuera de dominio, código duplicado, curso sin prerequisite cumplido).
Archivos/carpetas: backend/scripts_manuales/ (nuevo script de prueba)

Criterio de fase terminada (Definition of Done)

- Login, recuperación de contraseña y asociación automática de carrera/plan (RF-01 a RF-03) funcionan de punta a punta.

- Registro de perfil y validaciones de formato/prerrequisitos (RF-EST-01 a RF-EST-03) están completos y cubiertos por pruebas manuales.
- RF-PRF-01 (actualización de cursos por ciclo) implementado sin romper el historial de avance.
- Ningún endpoint de esta fase depende de datos hardcodeados fuera del esquema de la Fase 1.

FASE 3

Malla curricular, dashboard y perfil

Responsable: **Sebastián** | Rama: `feature/backend-core-app`

DEPENDE DE FASE 1 (mergeada). Puede avanzar en PARALELO con Fase 2. La Fase 4 depende PARCIALMENTE de esta fase (contrato de avance/perfil del estudiante).

Sebastián completa los datos que alimentan las pantallas que el estudiante ve la mayor parte del tiempo: malla curricular con avance, estadísticas de dashboard y gestión de perfil.

RF cubiertos: RF-04, RF-05, RF-06, RF-07 (malla curricular y progreso), RF-19, RF-20, RF-21, RF-22 (test de nivel y dashboard), RF-PRF-02, RF-PRF-03 (actualización de datos personales y credenciales), RNF-01 (usabilidad/navegación).

Subtareas (en orden)

- 1 Sincronizar la rama con main (post Fase 1) y completar en `routers/malla.py` el endpoint que devuelve la malla curricular completa organizada por ciclos (RF-04).
Archivos/carpetas: `backend/app/routers/malla.py`
- 2 Implementar en la respuesta de malla el estado de cada curso (aprobado / cursando / pendiente) por estudiante (RF-05), cruzando el historial de cursos aprobados de la Fase 2 con el catálogo de la malla.
Archivos/carpetas: `backend/app/routers/malla.py`
- 3 Añadir a cada curso de la malla el número de créditos y sus prerequisites (RF-06), reutilizando el modelo de prerequisites definido en la Fase 1.
Archivos/carpetas: `backend/app/routers/malla.py`
- 4 Implementar el cálculo de porcentaje de avance de carrera respecto al total de créditos del plan de estudios (RF-07), expuesto como endpoint reutilizable tanto por la malla como por el dashboard.
Archivos/carpetas: `backend/app/routers/malla.py`
- 5 Implementar el test de nivel inicial (RF-19): generación de un diagnóstico basado en los cursos que el estudiante ya cursó, y el endpoint que devuelve el resultado junto con una recomendación de ruta de aprendizaje (RF-20).
Archivos/carpetas: `backend/app/routers/dashboard.py`
- 6 Completar en `routers/dashboard.py` las estadísticas de actividad del estudiante (RF-21): inicios de sesión, exámenes rendidos y aprobados, avance por curso; y el filtrado por curso o periodo de tiempo (RF-22).
Archivos/carpetas: `backend/app/routers/dashboard.py`
- 7 Completar en `routers/usuarios.py` la actualización de datos personales limitada a nombres y apellidos (RF-PRF-02, sin permitir cambio de correo o código) y la actualización de contraseña con las condiciones de seguridad definidas en la Fase 1 (RF-PRF-03).
Archivos/carpetas: `backend/app/routers/usuarios.py`
- 8 Definir y documentar el contrato de datos de 'avance y perfil del estudiante' (forma de respuesta JSON de `malla/dashboard/perfil`) que Omar consumirá en la Fase 4 para personalizar evaluaciones y recomendaciones de IA.
Archivos/carpetas: `documentacion/backend/contrato-avance-estudiante.md` (nuevo)

Criterio de fase terminada (Definition of Done)

- Malla curricular con estados, créditos, prerequisites y % de avance (RF-04 a RF-07) completa.
- Test de nivel, dashboard de estadísticas y filtrado (RF-19 a RF-22) completos.
- Actualización de perfil y credenciales (RF-PRF-02, RF-PRF-03) respeta las restricciones de negocio (no cambia correo/código, valida seguridad de contraseña).

- Contrato de avance/perfil documentado para que Omar lo consuma en Fase 4.
- Rama sincronizada y mergeada a main antes de que Omar cierre la Fase 4.

DEPENDEN DE FASE 1 (mergeada). Puede iniciar tareas 1-3 en PARALELO con Fases 2 y 3, pero las tareas 4-8 (generación y reforzamiento con IA) requieren el contrato de avance/perfil de la FASE 3 ya mergeado. Es la ÚLTIMA fase en mergear a main.

Omar cubre el flujo académico más denso del backend: cursos y rutas de aprendizaje, generación de evaluaciones con IA (incluyendo RAG sobre exámenes pasados), reforzamiento personalizado y el banco de recursos complementarios.

RF cubiertos: RF-08, RF-09, RF-10, RF-11 (cursos y rutas de aprendizaje), RF-12, RF-13, RF-14 (recursos complementarios), RF-15, RF-16, RF-17, RF-18 (asistente de IA), RF-APR-01 a RF-APR-08 (módulos, formato de evaluación, configuración, reforzamiento con IA, semanas parcial/final, formato matemático), RNF-02 (rendimiento IA $\leq 5s$), RNF-08 (precisión de IA basada en RAG).

Subtareas (en orden)

- 1 Sincronizar la rama con main (post Fase 1) y completar en routers/cursos.py el acceso a cursos de todas las facultades en los que el estudiante está matriculado o desea explorar (RF-08), organizados en ruta de aprendizaje secuencial por semanas/temas (RF-APR-02, RF-09).
Archivos/carpetas: backend/app/routers/cursos.py
- 2 Implementar el marcado de temas/unidades como completados (RF-10) y el cálculo de porcentaje de avance dentro de cada curso (RF-11), persistiendo el progreso por estudiante.
Archivos/carpetas: backend/app/routers/cursos.py
- 3 Completar el banco de recursos complementarios: repositorio de evaluaciones pasadas (RF-12), videos de clases grabadas (RF-13) y acceso a material sin afectar el progreso de la ruta de aprendizaje (RF-14).
Archivos/carpetas: backend/app/routers/cursos.py
- 4 [Requiere contrato de Fase 3] Implementar la configuración previa de evaluación (RF-APR-04): endpoint que recibe observaciones del estudiante sobre metodología del profesor (lenguaje de programación, herramientas) antes de generar la evaluación con IA.
Archivos/carpetas: backend/app/routers/evaluaciones.py
- 5 [Requiere contrato de Fase 3] Implementar la generación de evaluaciones con IA (RF-APR-03, RF-17): alternativas de múltiple selección/única respuesta/verdadero-falso, 5 a 10 preguntas, panel de código ejecutable para el caso de programación, y selección de tema por búsqueda semántica (RAG) sobre exámenes pasados, usando rag/retriever.py y rag/generator.py.
Archivos/carpetas: backend/app/routers/evaluaciones.py, backend/app/rag/retriever.py, backend/app/rag/generator.py
- 6 Implementar el manejo especial de semanas parcial y final (RF-APR-07): extracción de títulos del resto de semanas desde la base de datos para consolidar la evaluación acumulativa, y soporte de formato matemático/científico en preguntas y alternativas (RF-APR-08).
Archivos/carpetas: backend/app/routers/evaluaciones.py
- 7 [Requiere contrato de Fase 3] Implementar el reforzamiento con IA (RF-APR-05, RF-15, RF-16): endpoint de consulta al tutor virtual apoyado en el contenido del curso y exámenes pasados (RAG), que recibe resultados de evaluaciones y avance del estudiante (del contrato de Fase 3) para dar retroalimentación y recursos de apoyo adicionales.
Archivos/carpetas: backend/app/routers/evaluaciones.py, backend/app/rag/retriever.py

8

Implementar el historial de conversaciones del estudiante con el asistente de IA (RF-18), y validar tiempos de respuesta del asistente contra RNF-02 (≤ 5 segundos) y precisión basada en contenido verificado contra RNF-08.

Archivos/carpetas: backend/app/routers/evaluaciones.py, backend/app/rag/generator.py

9

Verificación final de consistencia: probar de punta a punta el flujo generar evaluación → rendir → ver resultado → reforzamiento con IA, sincronizar la rama con la Fase 3 ya mergeada, resolver conflictos de contrato de datos compartido y solicitar revisión cruzada antes del merge final.

Archivos/carpetas: Checklist de QA funcional (documentacion/backend/qa-evaluaciones.md, nuevo)

Criterio de fase terminada (Definition of Done)

- Cursos, rutas de aprendizaje y recursos complementarios (RF-08 a RF-14) completos.
- Generación de evaluaciones con IA, reforzamiento y asistente virtual (RF-APR-01 a RF-APR-08, RF-15 a RF-18) funcionan de punta a punta con datos reales o de prueba.
- Tiempos de respuesta del asistente de IA cumplen RNF-02 y las respuestas se basan en contenido verificado (RNF-08).
- El contrato de avance/perfil de la Fase 3 se consume sin duplicar lógica de progreso.
- Checklist de QA funcional completado y adjunto antes de solicitar el merge final a main.

4. Cronograma sugerido (orden de ejecución)

El cronograma no asigna fechas fijas (a definir por el equipo), pero sí un orden de bloques de ejecución para evitar cuellos de botella:

Bloque	Quien trabaja	Que ocurre
Bloque 1	Luis	Ejecuta Fase 1 completa (esquema, seguridad, validaciones transversales). Los demás pueden leer los documentos de requerimientos y preparar entorno, pero no deben iniciar endpoints dependientes del esquema todavía.
Checkpoint	Todo el equipo	Luis mergea Fase 1 a main. Renato, Sebastián y Omar actualizan sus ramas locales desde main antes de continuar.
Bloque 2	Renato + Sebastián (paralelo) / Omar (tareas 1-3)	Renato ejecuta Fase 2 completa. Sebastián ejecuta Fase 3 completa. Omar adelanta las tareas 1 a 3 de la Fase 4 que no dependen de Fase 3.
Checkpoint	Sebastián	Sebastián mergea Fase 3 a main en cuanto está lista (no necesita esperar a Renato).
Bloque 3	Omar	Omar sincroniza con la Fase 3 ya mergeada y ejecuta las tareas 4 a 9 de la Fase 4.
Cierre	Todo el equipo	Merge final de Fase 2 y Fase 4 a main (en cualquier orden entre ellas). QA funcional cruzado de todos los RF contra los documentos de requerimientos.

5. Notas finales

- Este plan cubre exclusivamente el backend. Los cambios de frontend se gestionan en *UniVia_Plan_Rebrand_Frontend.pdf*.
- Los RF marcados como **MVP** en UNIVIA - REQUERIMIENTOS.pdf (RF-ALC-01, RF-ALC-02, RF-EST-01, RF-APR-01, RF-APR-02, RF-APR-03) y los marcados como prioridad **Alta** en Requerimientos_Plataforma_Estudiantes_UNI.pdf deben quedar cubiertos antes que los de prioridad Media/Baja dentro de cada fase, si el tiempo del equipo se ve ajustado.
- Cualquier desviación respecto a un RF de los documentos de requerimientos debe discutirse con el equipo antes de implementarse, no decidirse unilateralmente en la rama de la fase.
- Se recomienda una revisión cruzada (pull request review) entre pares antes de cada merge a main, incluso si la fase fue desarrollada por una sola persona.
- El módulo backend/app/rag/ (extractor, chunker, embedder, generator, ingest, retriever) ya existe parcialmente; la Fase 4 debe auditar su estado actual antes de asumir que cada pieza está completa.

